

A Review of AI-Assisted Impact Analysis for Software Requirements Change: Challenges and Future Directions

Ahmad Samhan
Software Engineering
Department

Faculty of Information
Technology

Zarqa University
Zarqa, Jordan

Ahmad.Samhan@zu.edu.jo

Suhair AlHajHassan
Software Engineering
Department

Faculty of Information
Technology

Applied Science Private University
Amman, Jordan

Su_hassan@asu.edu.jo

Sara Abu Dabaa't
Software Engineering
Department

Faculty of Information
Technology

Zarqa University
Zarqa, Jordan

Sara.abudabaat7@gmail.com

Ali Elrashidi
Electrical Engineering
Department

Faculty of Engineering
University of Business and
Technology

Jeddah, Saudi Arabia

A.elrashidi@ubt.edu.sa

Abstract— This paper presents a review study that explores the landscape and challenges of AI-assisted impact analysis in software requirements engineering. The growth of complex and change-prone software systems has highlighted the need for AI techniques—especially machine learning (ML) and natural language processing (NLP)—to be harnessed to improve impact analysis scalability and efficiency. Key challenges identified include data quality issues, scalability constraints, and ethical problems related to bias and accountability. To investigate these challenges and provide insights, a review was conducted across IEEE Xplore, ACM Digital Library, and Google Scholar, cross referencing between keywords that include concepts from the domains of “software requirements”, “impact analysis”, and “AI-Assisted approaches”. Inclusion criteria focused on studies that presents empirical data on AI-assisted impact analysis. This study highlights research gaps and presents insights for overcoming obstacles using emerging technologies such as edge AI in order to enhance post-impact analysis. Findings show that while AI improves traditional impact analysis, further work is necessary to deliver effective, ethical and practical implementations within evolving software contexts.

Keywords—Impact Analysis, Software Requirements, Artificial Intelligence, Natural Language Processing, Software Engineering, AI Challenges, Edge AI.

I. INTRODUCTION

Impact analysis is an important part of software development, helping to understand systems better, improve risk management, optimize resource utilization, and increase overall software reliability [1, 2]. Traditional change impact analysis methods like traceability matrices and expert judgment face various challenges when applied to large-scale and dynamic software environments, leading to the need of innovative solutions such as using AI-based techniques to improve scalability, accuracy, and efficiency.

Managing changes in software requirements is complex by its nature [3]. More recently, integrating artificial intelligence (AI) into the impact analysis approach has transformed the landscape by improving predictive capacity and producing efficiencies. While AI has been enabled, it has also brought with it more sophisticated tools and methods. Hurdles range from different risk analysis frameworks and methodologies [4]. Despite that AI-assisted impact analysis has shown encouraging progress, some research gaps have been found. These include a lack of empirical support and real-world validation, thorough frameworks for long-term evaluation and scalability, and little discussion of ethical concerns like bias, data privacy, and accountability. Additionally, their reliance

on small sample sizes, incomplete comparative study of AI tools, and disregard for transdisciplinary implications and scalability restrict the findings' generalizability. AI-assisted impact analysis has made advances but these tools struggle to empirically validate their assessments and often lack a clear ethical framework for independent review, important considerations for bias and privacy issues, and they are not scalable to large systems. The aim of this study is to recognize such challenges, provide insights for improving AI effectiveness in impact assessment, and to discuss potential solutions that technologies could offer, especially edge AI and quantum computing.

The nature of the review limits the generalizability of the findings concerning similar automated tools. First, it addresses gaps and suggests avenues for future AI-effects analysis. The objectives include looking at the current state of AI in software development, then heading toward problems in engineering, challenges that need to be met, and new areas of interest. This is followed by a structured paper review to evaluate how traditional and AI-led controls and propose a systematic way of dealing with the situation.

The remainder of the paper is organized as follows: **Section II** presents a background of impact analysis. **Section III** discusses the methodology of the study. **Section IV** investigates AI-assisted impact analysis. **Section V** highlights the challenges found based on the conducted review. **Section VI** provides insights related to future directions. **Section VII** concludes the work.

II. BACKGROUND AND LITERATURE REVIEW

A. Traditional Impact Analysis Methods

The ability of impact analysis in software engineering is understanding the effects of a change in a system. It enables developers to discover parts of the system that few change their behavior on the software requirements, policies, or regulations that might impact. Historically, this has involved a collection of manual or semi-automated methods, frequently depending on software engineers' knowledge. The major strategies that are commonly used are:

1) Traceability Matrix:

- What it does: A traceability matrix reveals the linkages between different elements within a set of development processes like test cases, codes, requirements, and design documents. It helps in determining to what extent any variations affect the

system components and whether it seems like all requirements are being met.

- How it works: Engineers update the matrix manually to test whether an aspect of code or design conforms with particular requirements. Updates: Good to know what is changing. When updating, it helps to know what needs to be changed.

- Limitations: Stabilizing the matrix manually can be expensive and bug-prone in large, complex systems [5]. Without regular updates, the information could quickly be out of date.

2) *Code Dependency Analysis:*

- What it does: This method focuses on the dependencies between the code parts to understand the impact of the change.

- How it works: Developers need to inspect function calls, interactions between code modules, and shared data manually or instrumentally to detect a potential impact.

- Limitation: This approach focuses only on the code and may overlook high-level policy or operational aspects and typically does not think about the downstream consequences for requirements or policy development [6].

3) *Expert Judgment:*

- What it does: To assess the effects of modifications, this manual method depends on the developers' expertise in examining system records and design.

- How it works: To determine how the system can impact change, the designer considers the system's history and structure.

- Limitations: There are mistakes in this process because it mainly depends on human judgment. The same change will result in different results from various engineers and is thus ineffective for large, complicated systems [7].

4) *Static Analysis Tools:*

- What it does: These tools run autonomously, performing regulatory analyses to discover pairwise interactions of the regulatory unit modules and for predicting effects.

- How it works: To determine how modifications might impact the code structure, control flows, data flows, and function calls are examined.

- Limitations: Static analysis is useful, but it just examines the code level and could overlook changes in requirements or design [8]. While operating, the system might overlook open connections. Traditional methods have some significant disadvantages, even if they have been used for a long period.

In a rapidly changing environment like Agile & DevOps, they first fail due to the high time and resource demands. Second, making poor choices is easy and can lead to expensive issues or system failures. Compared to old approaches, modern systems take longer to grow. These challenges highlight the need for more precise approaches than traditional ones, such as AI-enabled impact evaluations.

B. Key Concepts in Impact Analysis

Tracking and maintaining requirements during the software development process is essential for overall compliance, which can be achieved by using Requirements

tracking (RT) that facilitates developers in tracking changes more efficiently. There are a few major types of RT used. It guarantees that all requests will be executed continuously in their entirety. RT analyzes the links between requirements, regulations/policies and tests to assess how changes affect others and find a way to measure that. Moreover, RT also enhances satisfaction on business level by ensuring that the final product meets customer needs. Additionally, by guaranteeing that the finished product fulfills customer requirements, RT raises stakeholder satisfaction. Another way to ensure quality is to confirm that design plans and specifications adhere to the relevant standards. RT helps companies demonstrate adherence to specific industry standards' traceability criteria. Finally, by the explicit linkage of requirements to other components, the reduction of ambiguities, and the promotion of clearer collaboration, RT enhances communication between developers, testers, and stakeholders [4]

Demand variations might significantly affect the system, so it's critical to comprehend how they will impact various components. First, design modifications often result from changes in requirements, which may increase development time and cost. Requirement changes also necessitate code modifications, which might produce problems and necessitate thorough testing. Furthermore, there should be testing methods. Updated to reflect new requirements and increased rigor in the testing phase. All pertinent documents, including user manuals and design specifications, should be updated to prevent misunderstandings. Miscommunication can occur when it comes to change. Lack of stakeholder satisfaction, so they will be treated better. Also, a change of one module can inadvertently affect others, causing merger problems. Finally, the implementation of new requirements may impact budgets and schedules, placing additional strain on project management and resource allocation [4]

Software development and maintenance are continuous processes that need to be improved constantly to satisfy user demands, evolving technology, and shifting needs. Important elements of this process are iterative development methodologies such as Agile and DevOps, which focus on continuously improving software in response to user input and evolving needs CI/CD, or a continuous integration and implementation process, which guarantees timely updates and problem-solving. As the needs of users and stakeholders change, the software architecture must also change to remain useful and innovative. Regular maintenance should be performed after release to avoid system errors and failures, improve performance, and ensure safety. Software developers prioritize improvements based on customer feedback, keeping the program current. Technology costs must be controlled to equalize innovation and optimize the code for maximum performance. To give accurate information as the software evolves, all training materials and associated documentation should be updated. Finally, to maintain system performance under increasing workload, ongoing efforts are needed to optimize and scale the system as user demand increases [9].

C. Emergence of AI in Impact Analysis

Advances in machine learning (ML), natural language processing (NLP), and data analytics have boosted efforts to implement AI applications at scale in software development for routine tasks like static analysis and code rewriting. As far back as the early 2000s, ML was developed that used a rule-based framework to predict the potential impact of code changes. As NLP technology has evolved, AI tools have been able to handle natural language searches and documents and perform tasks such as requirements extraction and change management. Modern AI tools, including deep learning and transformer models such as Along with BERT and GPT, now have huge amounts of data. Complex relationships can also be processed, offering more sophisticated approaches to impact analysis and prediction. AI also uses graph-based algorithms to map dependencies and produce a diagram of system components, improving accuracy and efficiency [10].

As the complexity of software systems increases, traditional impact analysis methods face challenges, making AI increasingly valuable. Modern software systems are often large and distributed, with many interconnected modules, making manual analysis time-consuming and complex. AI automates much of this process, quickly analyzing dependencies and reporting overlaps, reducing human error and improving accuracy. In fast-paced environments like Agile and DevOps, AI real-time analytics interact seamlessly with CI/CD pipelines to enable dynamic change management and rapid response. AI systems can also scale, consuming large distributed systems, and handle unstructured data such as documents and user profiles. Predictive AI tools go beyond identifying effects—they assess the severity and risk of change and help developers prioritize important updates. By reducing human intervention, AI reduces errors in impact analysis, ensuring greater reliability, scalability, and better control of complex software systems.

AI tools developed in software engineering have evolved from basic automation to advanced AI systems capable of handling large and complex tasks such as impact analysis. AI's ability to manage today's increasingly complex systems a process increasing demand, improve accuracy, and provide real-time insights. It emerges from three boundaries, including human error, time consumption, and scalability issues. By automating processes and providing more reliable forecasts, AI helps manufacturers better manage variation, reduce the risk of error, and improve the system so all changes are effective.

III. METHODOLOGY

To gather relevant literature for this study, a systematic search was conducted in academic databases such as IEEE Xplore, ACM Digital Library, and Google Scholar, chosen for their extensive repositories of peer-reviewed articles in software technologies. Keywords like “software requirements change,” “machine learning in software engineering,” and “impact analysis challenges” were used to identify pertinent studies. The inclusion criteria focused on recent developments in AI and impact analysis, peer-reviewed research on impact analysis and software requirements technologies, and papers on challenges such as

scalability, accuracy, and data quality. In AI-assisted software analysis studies were excluded if they lacked empirical results, or comprehensive methodologies, or were not directly related to an impact analysis or software technology.

A thorough review of selected literature identified themes, gaps, and trends that are common in AI-assisted impact analysis. Particular attention was drawn to examine how AI can improve impact analysis, limitations of existing methods, and opportunities for future research, with particular attention to model accuracy, data quality, and scalability challenges.

IV. AI-ASSISTED IMPACT ANALYSIS: CURRENT APPROACHES

A. Natural Language Processing (NLP) for Requirements Extraction

Various NLP techniques can significantly enhance the process of requirement extraction and analysis in software development. A key technique is tokenization, where requirements are divided into smaller units called tokens, these tokens can represent keywords, variables, or symbols associated with requirements. This enables efficient text analysis and comprehension [11, 12], making it manageable for machine learning models [9]. Probabilistic models are also used in statistical methods for pattern recognition in large requirement archives, which are viewed as a set of signals for understanding and extraction [13]. Advanced language models, particularly transformer-based ones, help in requirement formulation, validation, and maintenance by comprehending the context and semantics of requirements. As supplemental requirements, NLP approaches can also be used to improve stakeholder communication and give developers concise explanations [13]. These techniques produce software specifications that improve analysis, extraction, and upkeep, which improves development processes and outcomes.

NLP is an important tool for making software more understandable by providing more ways to support clarity. For instance, NLP can employ methods such as Named Entity Recognition (NER) to focus on particular priority or recognize entities [13]. NLP capabilities can be used by teams to analyze requirements overlap [13], and pause searches as changes to one need could affect other needs. Automated adoption can also receive a boost by aligning on any vague or conflicting information in order to clarify the path forward. NLP is also helpful for instantly understanding documentation in larger and more complex projects [13]. Lastly, using charts or any other visual form of needs, NLP can also trigger conversations with stakeholders by making them understand what we mean. Applying these NLP techniques to comprehend requirements may lead the teams toward better software, communication and project [13].

B. Machine Learning Models

The two common approaches in machine learning are supervised and unsupervised techniques, widely applied in other domains including non-AI controlled impact analysis software requirements that apply to labeled manually

managed data sets at each point of the input with precise output labels. As such, the model might come up with new and unexpected predictions and find relationships between inputs. They are particularly useful when data is retained (historically) on the effects of known impacts, what changes were needed and how this affected people and places. In contrast, unsupervised research is the default for anonymous datasets they search through to make discoveries of patterns and relationships without defined outcomes [14-16]. This technique is particularly useful when we have low knowledge about the data structure of relevance since it can indicate patterns or clusters amongst dependencies or demands of a program. It also gives new angles on the possible outcomes that may not be evident at first try.

Both approaches offer valuable improvements to the software impact-analysis workflow, by using data insights to enhance human judgement ahead of decision-making in essentially complex systems. Although it defines all the aspects of analysis and reference in predicting the impact of software as predictive modeling, predictive analytics in software engineering is mainly done through AI models which depends mostly on machine learning and data-assisted methods because it provides a way for engineers to anticipate their perceived risks by ranking development tasks based on expected outcomes. This is essential for impactful analysis.

AI models (especially supervised learning-based models) learn from the change type history, characterized by their effects/impacts [1], such as induced bugs, performance improvements etc. In addition to that the state of art AI models of today also generate graphs representing dependencies between software components which can be used to predict how a local change on one component of the system can propagate and affect other dependent components [4]. This is particularly useful for complex systems with multiple interdependencies.

For dynamic environments like Agile and DevOps, AI integrated with Continuous Integration/Continuous Delivery (CI/CD) pipelines provides real-time predictive analytics for developers to make decisions quickly. This prediction includes not just the operational impact but also performance implications and security, inclusive of vulnerabilities as well as potential errors. Additionally, the AI model analyzes the complexity and risk associated with predicted outcomes, enabling teams to prioritize high-risk changes for scrutiny, giving low-risk changes priority and implement rapidly as AI models continue to evolve through feedback to become more accurate and reliable in complex contexts, changing software environments will become increasingly valuable tools for consumption change management [17, 18].

C. Case Studies and Real-world Applications

The case study of AI-supported architecture education demonstrates how AI technologies are integrated into instructional strategies to enhance student learning, idea generation, and resulting planning within the architecture curriculum. Findings show that AI significantly improved students' performance, enabled them to think creatively, and enhanced their multimodal problem-solving skills, while also preparing them for future work by providing AI tools they

saw well [13] Furthermore, these case studies provide useful insights into the quality of feedback, self-organization, and personalized learning experiences through AI-supported educational services and demonstrate the impact of AI on student engagement and its potential to facilitate learning activities [13]

The case studies highlight the many benefits of using AI in education, including improved programming skills, conceptualization, and policy implementation in models where AI helps. In addition, the integration of AI streamlined the process by automating technical tasks, allowing students to focus on the creative aspects of design. The majority of students had a positive attitude toward AI integration, and expressed a desire to use AI in the future, highlighting the potential of AI to improve transformation in fast-growing industries alongside data-assisted insights roles and personalized learning strategies, valuable for teachers and students Feedback was provided, which improved decision-making in the curriculum. Implementation of AI tools in the real world provided students with essential skills for their future work, gained a competitive edge in the construction industry, and provided insight into what can lead to better academic outcomes [13]

These cases demonstrate how AI supports impact analysis and AI tools in educational settings for significant benefits, including increased academic achievement, increased productivity, and innovation They also demonstrate long-term positive impacts, such as providing necessary AI to skilled students to prepare you for future work.

D. Benefits of AI in Impact Analysis

By automating aspects of AI data analysis and decision-making, software significantly reduces the manual effort required to predict the impact of required changes [14] Natural language processing (NLP) enables developers and analysts to specify requirements and ask questions of influence in plain language, eliminating the need for complex rules [14] This capability democratizes the impact analysis process, enabling even non-experts to engage with the tools effectively. Additionally, AI can automate data processing, and manage the selection, modification, and analysis of software metrics and dependencies, which are essential to understanding the ripple effects of change by automatically generating the necessary code for research, AI reduces the time and cognitive load of the user [19, 20].

AI accelerates workflow by automating repetitive and routine tasks, such as dependency analysis or change tracking, so researchers can focus on the interpretation of results and high-level decisions It can also help debug and generate reports, ensuring accurate analysis and freeing up developers to focus on the more complex and creative aspects of software development [14] Furthermore, AI tools can suggest new ways to analyze change impacts, expand the analysis, and increase decision-making efficiency All of these capabilities combine to make impact analysis more efficient and effective, reducing manual software burden groups, and the accuracy of predictions increasing.

AI-enabled methods offer significant improvements in accuracy over traditional manual methods, especially in predicting the impact of changes in software requirements. The main advantage of AI is its ability to make predictions with confidence values [21]. These confidence levels provide decision makers with an indication of the reliability of the predictions, increasing the overall accuracy of the impact analysis. If AIs are designed to provide these confidence levels and the decision maker's confidence level meeting, a process known as human conferences where it enables teams to make informed and realistic decisions about software changes [22, 23].

Another important technique is multi-calibration, which ensures that the confidence values are reliable in different subsets of data and improves decision accuracy for different components of the software module [24]. This statistical approach yields predictions of greater complexity, thereby reducing overconfidence or underestimation of the specificity of regions in the database [25, 26].

This improvement in accuracy is important in the case of complex software systems, where manual methods often struggle to account for changes in complex dependencies and unexpected effects if changes occur. AI-assisted impact analysis has many advantages. One key advantage is the automation of data processing. On the other hand, the things AI tools can do for you, such as data extraction, dependency mapping, and change tracking [21]. In addition to allowing developers to focus on complex aspects of analysis, this automation gets exhausting when, with their own hands, these materials for discovery and analysis are greatly reduced.

AI facilitates quick query processing, allowing developers to ask impact questions in natural language and get immediate results without the need for extensive manual processing [14]. Additionally, AI supports streamlined business processes by integrating various steps in the impact analysis process, such as retrieving relevant information, analyzing regulatory underpinnings, and generating impact reports, all on coordinated systems, thereby reducing the time and effort normally required to switch between different tools and crafts for production [14]. AI's ability to identify and correct errors in the inspection process further accelerates productivity by quickly identifying potential problems and suggesting fixes, enabling faster iterations and changes [14].

AI also looks for alternatives, offers suggestions that developers have not considered, and enables faster, more informed decisions without the need for time-consuming analysis. Besides, reporting effectiveness is another time-saving feature, as the AI tools are capable of detailed impact analyses on self-reporting and visualization. Save and collect manually. Over time, as AI systems learn and adapt to work that meets its specific needs, the analysis becomes more sensitive, resulting in increased efficiency.

By automating routine tasks and enabling rapid feedback, AI-enabled impact analysis reduces the time required for manual tasks, enabling development teams to make faster and more informed decisions about changes in software

requirements, ultimately improving project efficiency and results [14].

V. CHALLENGES IN AI-ASSISTED IMPACT ANALYSIS

A. Data Quality

Finding the right information is a big challenge when using AI-assisted impact analysis in software development. The success of AI models depends on removing several significant barriers. First, data privacy and security are the primary concerns, especially when dealing with proprietary software and user data. Following the law and moral principles can limit access to data and have an impact on how well AI-assisted impact evaluations work [27]. Second, information needs to be representational of the entire software system, including all of its parts. Inaccurate analysis might arise from skewed forecasts caused by biased or inadequate data. Third, in vast software ecosystems, the linkage between tools and systems is crucial. The absence of established formats and protocols complicates data integration and hinders AI's ability to make precise predictions [27]. Fourth, price and scalability are crucial factors to take into account because gathering and handling vast volumes of data can be costly, particularly for smaller businesses utilizing AI solutions that are readily scalable [27]. Finally, the success of AI models depends on the quality of the data.

Inaccurate forecasts and deceptive developers regarding the actual effects of software modifications might result from low-quality data. Software engineers, data scientists, and legislators must work together to create guidelines, protocols, and standards for efficient data gathering and the application of AI to impact analysis to meet these problems.

It is a very expensive and resource-consuming process to clean and prepare data for AI-based impact analysis, which needs multiple important steps to ensure the success of the AI model in predicting the impact of changes in software requirements. This process ensures that the data collected is appropriate, comprehensive, and representative of the software project to accurately inform the AI model [27].

The next crucial step is data cleaning, where you need to differentiate and correct errors or irregularities in the data structure. Problems that we usually experience are missing values, for example, an incomplete record of dependent code, or modifications that can compromise forecasts if not addressed. Techniques like partial data removal or imputation can be applied. To prevent unnecessarily influencing model performance, noteworthy elements in the data, such as extreme instances of software modifications that do not follow a particular pattern, should be carefully examined. In addition, they ought to eliminate duplicate entries to guarantee that the dataset appropriately captures distinct observations [28, 29]. The data conversion process includes converting data to a format for analysis, like normalizing numeric values or encoding categorical variables like error severity, which may be able to process only specific types of data too sensitive internally based on how the categorical are scaled or their potential differences.

Writing specifications where past modifications are noted and categorized based on their effect on the system may be one way to monitor a curriculum. According to systems expertise and previous research, viewing and classifying data can require more manual effort as there are additional challenges to integrating data that may require human expertise, especially when it comes from multiple systems and sources in the 19th century. For the predictions to be accurate, the quality and accuracy of the data must be maintained. AI models must be trained on high-quality data, which requires quality control methods including statistical verification of data creation and selection [27].

Documenting end-user analysis and data generation accurately for developers and stakeholders is essential for transparency and repeatability when data scientists, software engineers, and information professionals work together to prepare data for AI models in impact analysis. Doing it takes a lot of work. AI models are accurate in this investment is needed in the field to ensure availability, capability reliability, and can provide useful insights.

B. Interpretability and Explainability

Understanding how AI models make decisions for various reasons is often difficult, especially when the models are complex. Model complexity is an important issue, especially in deep learning models with multiple layers and multiple dimensions. This creates a "Blackbox" effect, making it difficult to monitor the effect of certain inputs, such as rule variables or policy dependencies, on forecasts [30]. Furthermore, it can be difficult to understand how small changes in one area can have significant and unexpected effects because these models often problematically assume nonlinear relationships between input variables and complex feature interactions when another comes along.

It turns out that the non-transparency of many AI models further complicates matters because they rarely explain choices [30], especially when it comes to important software decisions or can undermine trust when they manifest as biases in some of the training data [30]. As software improves, AI models are often updated or retrained with updated data, which can change decision-making processes and try to understand how and why predictions are made at the bottom of the table. Emphasizes the need for strategies, that keep AI efficient, accurate, and considered for many reasons, the unpredictability of AI systems can severely hinder their adoption.

First, trust problems arise when users rarely trust AI systems that act as "black" with unclear decision-making processes [24]. Stakeholders may question the reliability and accuracy of AI models, especially in high-stakes decisions, if they cannot understand how predictions are made. Second, accountability becomes difficult without transparency, as it is challenging to trace the source of errors or biases in AI predictions [24]. Third, there may be user acceptance, especially among non-technical users, when the role of AI systems is unclear, reducing user comfort and trust [24]. Fourth, Transparency is critical information for monitoring, identifying, and reducing bias. It is difficult to assess the quality of a system without insight into its implementation,

which can lead to public backlash and discredit. Finally, lack of insight inhibits innovation, as it limits progress categories from understanding, maintaining or optimizing AI technologies [24].

C. Accuracy Limitations

AI models, particularly those applied to impact analysis or requirements management in software development, can face several performance challenges. First, ambiguous requirements can lead to misinterpretation by AI models, causing incomplete or incorrect impact predictions when the language lacks clarity [9]. Second, because domain-specific language can be difficult for AI models to grasp, especially when technical jargon from specialist fields is utilized, sophisticated training on domain-specific data is required [13, 31, 32]. Third, the quality and quantity of training data is another consideration when modeling AI. If the data collection is small, local or unrepresentative with respect to the actual workload, then it will adversely limit how well a model can generalize to unseen (or unexpected) requirements [13]. Fourth, unless stated otherwise this makes it much easier for AI models to incorrectly forecast the dependent variable than others in case of high model complexity and dissimilarity between software requirements. Fifth, in changing environments of agile firms, AI and advanced data analytics will become obsolete: these tools require periodic monitoring of their effectiveness so that new models be produced with an adequate frequency [13]. A sixth ethical and legal challenge that can arise in the course of training AI models is the handling of copyrighted material which, if violated, may lead to litigation.

A lack of certainty affects the patience and trust of stakeholders. These constraints highlight the need for real human supervision, continued policy and scrutiny before using AI to drive research and or impact software engineering [13]. AI model training sometimes has very tight restrictions which can impact performance. Overfitting occurs when a model struggles observing data because the training data contains noise and unwanted information in addition to observations. Methods such as regularization and cross-validation often help to mitigate this problem. On the other hand, when a model is too poor, it leads to underfitting [33], [34].

This can be addressed with techniques such as oversampling, and weighted loss functions. Another common issue is the data – either come without enough data for training, especially when dealing with more complex models such as deep learning setups which require large amount of database to cope with the complexity of the input data. This has a greater effect determined that feature selection and data cleansing decreases. If incorrect input variables are picked, poor specification engineering can result in either an underfitting or excess complexity or unnecessary noise. However, model optimization may be hampered due to long training times (especially true for deep learning models). Although Health type-wise distributed training or special hardware like GPUs can mitigate this. Finally, constant retraining and the continuous learning process is needed due to model drift since data distribution changes over time, degrading the performance of the model spontaneously. To

address these challenges, continuous retraining and learning processes are required. It is important to build a reliable AI model that works well in practice, requiring consideration of model design, data quality, and training procedures. However, all of these steps are necessary but not sufficient.

D. Model Training and Maintenance

Building AI models for impact analysis in software development requires huge computing resources for several reasons. Modern AI models, especially large initial models such as GPT-4, require large amounts of memory and processing power in training and simulation, making them expensive to train and potentially inaccessible to smaller organizations [27]. Careful management of training data can reduce computational costs by ensuring that smaller samples perform compared to larger samples, improving productivity without degrading performance [27]

Models that incorporate execution-aware pretraining, such as those designed to understand complex software logic with execution traces, also require additional resources but there is growing interest in developing smaller, more cost-effective models designed for software engineering tasks. It can reduce resource requirements by focusing on code interpretation and impact analysis [27]

AI models that incorporate continuous learning and feedback mechanisms increase computational load due to the need for continuous data processing, and further training is therefore required, although it improves the accuracy of the model over time [27]. Although AI modeling requires important features for the impact analysis, e.g. optimizing sizes, improving data security, and developing more efficient training methods can help reduce these requirements and improve the quality of AI solutions.

Constantly evolving AI paradigms in software development to keep pace with the rapidly changing landscape of coding practices, technologies, and emerging challenges dynamic nature of software development, with frequent programming languages, frameworks, and methodologies updating the project, requires AI to be able to code well up to current standards [27]. Models need to be adapted. The quality and relevance of the data used to train these models are important, needing regular updates to reflect evolving regulatory trends [30]. Ongoing learning processes enable AI models to evolve to ensure new data incorporation, monitor compliance quality, and align with industry trends [30]. As new challenges emerge, such as security vulnerabilities and operational challenges, AI models are also developed to address these issues through continuous analysis and algorithm modification [30].

AI models are able to help alter social norms and as its outcome, practitioners should implement the ethical factors of their work from biases in training data while also weighing up whether an AI-based decision is always legitimate. In order for paradigms in AI to remain viable, satisfy new challenges and the most demanding problem-solving capabilities during the software development process, they need to adapt [35].

E. Scalability and Integration

In integrating AI tools into large complex software projects there are several significant hurdles that developers need to get over first. One important challenge, incompatibility of generative code with the project requirements as engineers have low control over it. This makes it challenging to simply improve old code with AI-generated code [36]. Most large projects may involve complicated contexts that could be challenging for AI to infer intelligible outputs without sufficient contextualization. This may lead to obsolete or undecipherable code [36].

Issues related to AI-generated code's quality and reliability, such as syntax mistakes or coding standard violations are time-consuming to review; these processes may slow down productivity. In addition, the fact that classical programming processes are not compatible with AI technologies is a problem — it leads to a "copy-and-paste" method, which lowers productivity [36]. When searching for common improvements that are applied to large projects, many change cycles may be necessary and without gaining effective results. Using AI techniques replication can be somewhat resource intensive in this case [36]. Adopting AI technology is not simple due to the complex ethical and legal issues of the developers, for example copyright privacy problems. AI technologies need to be tailored and constantly honed so that they can respond to the demands of complex software development ecosystems, in order to overcome these hurdles [36].

Integrating AI into existing business processes, especially requirements engineering (RE), presents many challenges. AI paradigms, large language models (LLMs), often struggle with stringent, domain-specific requirements, especially in security-critical environments, leading to biased or inconsistent results that hinder decision-making in RE processes [37].

Research focuses on the early stages of RE, such as requirements-provision and analysis, while neglecting later stages, such as requirements management and development, limiting the overall impact of AI throughout the RE lifecycle [37]. Besides, decisions generated by AI often lack translation and traceability, which is necessary to maintain reliability and accountability for unlimited integration [37]. Perhaps research designs with existing ones will not adequately assess AI performance in RE, requiring new metrics that evaluate results based on logic, and consistency [37]. Ethical and legal challenges, including concerns about bias, data privacy, and decision-making accountability, add challenges to AI integration [37]. At the same time, the need for human and AI collaboration requires easy communication between engineers using AI. The high technical costs and resource limitations that allow customization of the design [37]. Make the adoption of AI difficult, especially for organizations with limited resources. Furthermore, AI models often struggle with real-time processing and repeatability, making their integration into dynamic business systems difficult [37]. Addressing these challenges is critical to ensuring delivery of AI will effectively and ethically integrate into current business processes.

F. Ethical and Bias Considerations

Biased data in AI systems can lead to biased results through several mechanisms. First, if the training data are not representative of real-world situations, the AI model can learn to accommodate those biases. This has serious implications; a facial recognition system that was trained on a population largely of one ethnicity or gender may function extremely poorly and provide biased outcomes when used in scenarios with diverse demographic groups.

AI models also lead to broader cultural assumptions, having been trained on historical contexts that exhibit such [38]. For instance, a bias remains present if an outdated recruiting algorithm that previously discriminated against disadvantaged groups is still identifying these segments automatically [38]. If, in designing the model, criteria for optimization include certain outcomes without consideration of fairness, then algorithmic bias could be built into it. For this reason, bias correction is challenging, especially in complex systems [38].

Feedback loops can also bias AIs. For example, predictive police programs that target neighborhoods based on biased data produce more skewed data, reinforcing biases [38]. Lack of diversity development teams fail to measure how AI models impact specific populations, resulting in unfair and unreasonable biases [38]. Using bias detection and correction techniques to provide estimates and results that appear in AI systems is effective, providing representative training types for information. Both policies that encourage and justify AI behavior through shared responsibilities are needed to remove these barriers.

Everyone from developers to organizations, legislators, and users is accountable for making sure that AI is used appropriately across the board. AI features, especially those that are complex, like large language models (LLMs), deserve to be held accountable, meaning human oversight is needed. AI can perform many tasks but the onus remains upon humans, especially as sophisticated disciplines such as software engineering are involved; errors here could have catastrophic consequences [39].

Property rights or copyrights of AI-generated content, including other relevant legal considerations; and Stakeholders need to define the usage restrictions regarding how their work can be used by others. Data security and privacy are equally important, as AI models might inadvertently expose sensitive personal information [39]. This calls for strong data protection measures [39]. The new generation of AI technology is posing challenges that are not straightforward to answer and would need research concentrated on emerging ethics or a study of scenarios no one has faced before [39]. Appropriate AI practices require cooperation across stakeholders to foster the ethical, responsible, and transparent use of AI technologies.

VI. FUTURE DIRECTIONS

A. Advancements in AI Models

Impact analysis in software development and requirements engineering could be greatly enhanced by deep

learning. Deep learning models outperform manual methods in automated requirements analysis, which is essential for managing massive document volumes, identifying important features, dependencies, and change effects, and enhancing implementation [13]. Furthermore, predictive modeling with deep learning enables models to evaluate how changes in demand will affect system performance, dependability, and user satisfaction by looking at historical data and helping stakeholders make informed decisions [13]. Deep learning in natural language processing (NLP) can improve unstructured requirements analysis, extract meaningful insights, identify ambiguities, and satisfy the requirements for better impact analysis [9]. Another advantage is dependency analysis, where deep learning models can provide relationships between software components and show how changes in one area will affect others, especially useful in complex systems with complex dependencies [9].

Furthermore, deep learning facilitates the analysis of emotions and stakeholders, processing comments and comments to assess emotions and potential concerns, which is important to understand how those who implement and stakeholders understand the changes. They also enhance visualization and reporting capabilities, as deep learning provides reliable visual effects and consumers a sense of the role, which makes it easier for stakeholders to understand the implications of the change. More importantly, deep learning models can constantly learn and adapt, generating new data and requirements will evolve, especially in fast-paced development environments where requirements change frequently. Finally, deep learning can be combined with other AI techniques such as reinforcement learning and genetic programming to analyze situations, refine needs, and optimize the impact analysis process [13].

Deep learning has the potential to transform impact analysis by automating key processes, improving predictive accuracy, and increasing understanding of requirements and interdependencies, leading to improved decision-making and good results in software projects. Deep learning provides the necessary capabilities to enable software development processes to change requirements, feedback, and conditions. One key advantage is real-time data processing, allowing models to respond immediately to changes, such as new or modified requirements, making them particularly useful in agile environments where continuous adaptation is essential [9]. Additionally, continuous learning mechanisms enable deep learning models to adjust their parameters and performance as new data becomes available, ensuring they remain relevant and accurate as software evolves [9]. Feedback between users, developers, and the deep learning model further improves flexibility, as the feedback helps refine the model algorithm and provides feedback on the project objectives and evolving user needs [9].

Deep learning models can acquire contextual knowledge by searching for patterns in historical data and identifying contextual cues, enabling prediction of potential changes and management of frequent variable priority. Dynamic requirement prioritization is an added benefit. Additionally, automated impact analysis enables deep learning models to analyze how rapidly changing one part of the system impacts

other elements, giving developers real-time insights on how to make appropriate decisions.

Integrating deep learning into DevOps practices further supports dynamic development, especially in Continuous Integration and Deployment (CI/CD) environments, where automation of testing and deployment takes care to realize that the changes will be used correctly [9]. Finally, the scenario simulation deep learning model is based on the simulation of various outcomes on possible changes to be made, enabling developers to plan well and adjust for future changes. Deep learning enables software development through real-time implementation, continuous learning, feedback mechanisms, and contextual understanding. This capability allows developers to react quickly and efficiently to the needs of ongoing developments and ensures that software remains quality and relevant in a fast-paced development environment [40].

B. Enhancing Human-AI Collaboration

Combining human knowledge with AI capabilities is essential to improve decision-making processes and ensure that AI systems align with human values and needs. Human and AI collaboration plays an important role since domain experts, like doctors, financial researchers, or engineers, introduce contextual AI predictions that may be lacking. By integrating human insights, more nuanced and informed decisions can be made, ensuring that factors are considered important [21]. Furthermore, measuring trust and confidence between human decisions and AI systems enables greater collaborative decision-making. This ensures that human knowledge complements AI capabilities, resulting in improved decision-making [24].

The concept of multi-scaling, which ensures that AI systems provide reliable values, help decision-makers gain confidence, and use AI insights effectively, further enhances this interaction [24]. This is supported by empirical research, which shows that human decision AI has high reliability levels. Or maybe it just works better than human intelligence. [24]. Since people can support AI systems and learn from AI discoveries to change their minds, learning and adaptation are necessary. In addition to increasing decision-making efficiency, this iterative process accelerates the development of artificial intelligence and human intelligence [24].

Moreover, AI combined with human intelligence is the best solution to deal also with ethical issues. It ensures that decision-makers can hold AI-generated insights accountable in moral as well as social aspects, giving them the power to pass judgments on more than just facts but also concerning humanity. Good decision-making is enhanced, trust is built, and ethical standards and human judgment are represented in the decisions being made by combining humans plus AI.

The integration of human intelligence and AI enhances decision-making power by utilizing the best combination of these forces to develop more trust while keeping decisions ethical as well as respecting humanity.

For tools powered by AI to be adopted in industries like software engineering, healthcare, and finance, which are fairly complex other than the fact that they also require

sophisticated technology, these products need widespread adoption by those who know how, will, or even bother with it. A human interface would allow such AI tools to be used by individuals who may not have a strong technical background for them to leverage it within the decision-making process. This access & ease of use is important for several reasons:

1) Access by non-experts: AI programs must be sufficiently interactive for managers and front-line employees to interact with and understand the consequences of AI. For example, clear visualizations and straightforward explanations in AI-supported impact analysis can provide insights that even project managers without technical knowledge of AI can learn.

2) Building an AI Practitioner Community: These tools are quite helpful in allowing non-specialists to reach the requirements anytime in the future. In areas such as the medical domain where some level of a priori knowledge is required but there are still constraints to the use of AI, user-friendly interfaces allow for greater participation. This aids in situations where AI suggestions are made, aided by medical practitioners such as doctors and nurses.

3) AI infographics: Such contradictions between the tasks of the system and its forms appear due to the underdevelopment of proper means. AI technology should actively use mathematical and statistical information visualization through charts and graphs to facilitate the understanding of data as a task for its users. AI models that provide natural language explanations are examples of interpretable AI that give the system and its users language necessary for understanding the concepts about the AI's predictions.

4) Customization: Designing sophisticated systems with full deepening to prepare for users' needs is not viable. To do so, users should be able through interactive and flexible interfaces to change parameters or try different alternatives. This enables clients to configure AI-based applications based on their requirements and objectives, thereby enhancing the system's adaptability and situational awareness.

5) Abet collaboration: The main disadvantage due to the gap between development and business people is the need to employ a wide skill set. Done effectively, this utilizes AI-enabled intelligence and nurtures seamless cooperation between management, developers, and QA teams [41, 42], [43].

Decrease cognitive burden and improve productivity: Providing information in a clear and easy-to-understand manner enhances the subjective user experience. However, it reduces cognitive efforts, speeds up decision-making processes, and possesses seemingly simple but well-designed features like minimum adjustment. This makes users not to be bogged down by jargons or excessive technicalities in everyday use of the product [44, 45].

User-friendly AI interfaces are critical to ensuring that AI tools are more accessible, effective, and acceptable, allowing different users and AI to interact in ways reasonable and productive.

C. Explainable AI (XAI)

Ensuring that AI decisions are understood, often referred to as "explainability" or "transparency," develops the trust and acceptance necessary for the successful integration of AI systems across industries as users understand how AI decisions are made, reducing skepticism, particularly in high-stakes sectors like healthcare, finance, and law, as the interpretable form of AI enables stakeholders to understand and take responsibility for AI-assisted outcomes, which is important in environments where transparency is key [46]. Also, error identification becomes easier, as users can follow AI logic and identify any errors or biases, improving system reliability [46]. Ethically interpretable AI ensures that decisions can be tested to prevent biased or unfair results; this is an important consideration as AI plays a larger role in life [46]. From an ethical perspective, explainable AI ensures that decisions can be scrutinized to prevent biased or unfair outcomes, an important consideration as AI assumes a greater societal role [46]. Finally, user empowerment takes effect when users can understand AI insights and integrate them into their processes to make informed and effective decisions [46]. To effectively use AI to build trust, ensure accountability, identify errors, address ethical issues, and meet legal requirements, translation is also important to empower users.

Several approaches have been implemented to improve the interpretability of AI systems, ensuring that users, regardless of their technical skills, can understand and rely on AI-assisted decisions. Developed example agnostic definitions to describe any AI system, regardless of complexity.

The opposite explanation looks for changes in economic data that will produce new results, which is particularly useful for understanding pattern sensitivity and finding new scenarios. In areas such as economics, this can predict changes that will mean loans have been approved rather than rejected. Prominence maps provide insight into areas of image, text, or time series data that had the most influence on the model's decisions and provide intuitive knowledge of how the model "sees" the data.

Rule-based decision trees and models naturally decompose decisions into simple and logical sets of rules that allow for high-level interpretation. Although these models are not as efficient as complex models, they can be used as interpretable proxies for specific, simple situations. Interactive languages allow users to engage AI systems directly, modify inputs, and analyze scenarios to understand the behavior of the model, which is especially valuable in critical industries such as healthcare and software engineering, where human recognition is important.

These approaches give obvious things, helping users better understand, trust, and act on AI, which is required to be adopted in the critical areas of statistics and interpretation.

D. Standardization and Best Practices

Impact evaluations are increasingly including artificial intelligence; thus, appropriate design and material techniques are required.

Specific metrics that the AI models should be measured against for software impact, accountability of AI usage, transparency level, equity and ethical compliance level within the AI model generated value chains as well as standard(s) for data to enable deployment of those systems in any software platform via integration.

To enable the appropriate use of AI tools in impact evaluations, it will be necessary to establish industry-wide standards. This set of best practices should involve the establishment of data governance protocols for how to view, store and maintain datasets, collaboration between human experts and AI systems, and the building and testing of models using appropriate methodologies to ensure that AI aids rather than supplants human judgment. AI cannot inherently create bias, so there are needs to be ethical structures put into place for the system to run transparently and responsibly.

Through these regulations and guidelines, agencies can check whether AI technologies are yielding quality outcomes that are transparent to be trusted and support a common framework for measuring the impacts that AI expands. This improves the decision-making process in software engineering.

E. Emerging Technologies

This area could experience a revolution with the advancement of quantum computing, given that AI-enabled impact analysis will considerably increase processing capacity and productivity. Quantum computing helps AI models to analyze massive amounts of data more efficiently and identify complex relationships at speeds we never imagined before [47]. Quantum computers could also change/fit AI to open up in depth study of rules and systems [48, 49]. While still in its early days, quantum computing when evolved can change the landscape of software engineering by changing how we apply artificial intelligence (AI) technologies in Software Engineering. Edge AI has unique advantages for impact analysis because it employs AI locally, rather than in centralized cloud servers. The attributes of Edge AI that are conducive to automation ensure real-time discovery of software updates without a sprawling cloud infrastructure.

In fields like embedded devices and the Internet of Things system, where quick decisions are necessary, this makes it possible to detect and forecast the effects of software changes more quickly. In addition to this, Edge AI reduces reliance on cloud infrastructure that makes impact AI driven analytics more adaptable and economical for companies with limited computational resources. Additionally, local storage of sensitive data improves data privacy and quality by lowering the dangers of internal data distribution on centralized systems [49-51].

VII. CONCLUSION

This paper examines the development and benefits of AI-assisted impact analysis, focusing on how AI improves accuracy, reduces human labor, and provides scalable solutions for managing complex software systems through techniques integrating machine learning and natural language processing (NLP). Program development in general and

device communication in particular can be accelerated by humans.

Integrating AI into impact analysis has a big impact on software engineering. Development teams can work more accurately and efficiently by automating the multitasking process of assessing the impact of software modifications. AI technology also makes risk management easier by guaranteeing that big, complicated systems may grow without compromising performance or efficacy.

As AI technology develops, more research and advancement in AI-supported effect analysis are essential. Emerging technologies like edge AI and quantum computing present numerous opportunities to enhance the precision, speed, and application of AI tools. Furthermore, developments in semantic AI and modeling will be crucial to maintaining openness, confidence, and the ethical application of AI in software development.

The integration of AI into impact analysis is a major advance in software technology. Artificial intelligence can dramatically improve the management and design of software systems through increased accuracy, complex automation, and human brains with AI-enabled impact analysis. An essential tool to overcome today's increasing software challenges while advancing AI technologies.

VIII. REFERENCES

- [1] S. Alyahyan *et al.*, "Software reliability assessment: An architectural and component impact analysis," *Tsinghua Sci Technol*, pp. 1–19, Jun. 2024, doi: 10.26599/tst.2024.9010101.
- [2] A. Samhan, M. Odeh, M. Kossmann, and A. G. Tbakhi, "Business process architecture-driven change management applied to a cancer care organization in a systems of systems context," in *Proc. 1st Int. Conf. Cancer Care Informatics (CCI)*, 2018, pp. 205–214.
- [3] A. Samhan, "OntoSoS. BPA. ChM: A semantically-enriched and business process architecture-driven framework for change management in 'System of Systems' context," Ph.D. dissertation, Univ. West of England, 2020.
- [4] M. E. Walsh, "Towards Risk Analysis of the Impact of AI on the Deliberate Biological Threat Landscape," Jan. 2024, [Online]. Available: <http://arxiv.org/abs/2401.12755>
- [5] Subik, Pokharel., Hassan, Reza. (2019). 4. Toward the Design and Implementation of Traceability Engineering Tool Support. *Journal of Software Engineering and Applications*, doi: 10.4236/JSEA.2019.126015
- [6] Detecting Software Security Vulnerabilities Via Requirements Dependency Analysis." *IEEE Transactions on Software Engineering*, undefined (2022). doi: 10.1109/tse.2020.3030745
- [7] Pedro, Faria., Eduardo, Miranda. (2012). Expert Judgment in Software Estimation During the Bid Phase of a Project -- An Exploratory Survey. 126-131. doi: 10.1109/IWSM-MENSURA.2012.27
- [8] Anjana, Gosain., Ganga, Sharma. (2015). Static Analysis: A Survey of Techniques and Tools. 581-591. doi: 10.1007/978-81-322-2268-2_59
- [9] N. Hashim Al-walidi, N. Ramadan Darwish, and A. Hussein Zolait, (2024) "Requirements Traceability Approaches: A Systematic Literature Review."
- [10] L. Reimann and G. Kniessel-Wünsche, "Achieving guidance in applied machine learning through software engineering techniques," *Proceedings of the 42nd International Conference on Software Engineering*, 2020, pp. 123–133. DOI: 10.1145/3397537.3397552.
- [11] F. Nazir, W. H. Butt, M. W. Anwar, and M. K. Khattak, "The Applications of Natural Language Processing (NLP) for Software Requirement Engineering - A Systematic Literature Review," in *Lecture Notes in Electrical Engineering*, vol. 488, Springer, 2017, pp. 643–651. DOI: 10.1007/978-981-10-4154-9_56.
- [12] A. A. Sabriye and W. Zainon, "A framework for detecting ambiguity in software requirement specification," *2017 IEEE Conference on Information Technology, ITCC 2017*, 2017, pp. 1–6. DOI: 10.1109/ICITECH.2017.8080002.
- [13] M. F. Wong, S. Guo, C. N. Hang, S. W. Ho, and C. W. Tan, "Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review," Jun. 01, 2023, *MDPI*. doi: 10.3390/e25060888.
- [14] A. Darvishi, H. Khosravi, S. Sadiq, D. Gašević, and G. Siemens, "Impact of AI assistance on student agency," *Comput Educ*, vol. 210, Mar. 2024, doi: 10.1016/j.compedu.2023.104967.
- [15] V. Musco, A. Carette, M. Monperrus, and P. Preux, "A Learning Algorithm for Change Impact Prediction," in *Proceedings of the 38th International Conference on Software Engineering*, 2015, pp. 1–10. DOI: 10.1145/2896995.2896996.
- [16] N. Yue and Z. S. Khodaei, "Assessment of impact detection techniques for aeronautical application: ANN vs. LSSVM," *International Journal of Structural Health Monitoring*, vol. 12, no. 2, pp. 165–178, Nov. 2016. DOI: 10.1142/S1756973716400059.
- [17] J. Yu, F. Ding, C. Guo, and Y. Wang, "System load trend prediction method based on IF-EMD-LSTM," *International Journal of Distributed Sensor Networks*, vol. 15, no. 8, Aug. 2019, pp. 1–10. DOI: 10.1177/1550147719867655.
- [18] C. Feng, Y. Li, and T. Wang, "A CI/CD Pipeline with Predictive Analytics for Agile Development Environments," in *Proceedings of the 2018 International Conference on Software Engineering*, 2018, pp. 155–162. DOI: 10.1109/ICSE.2018.165.
- [19] M. Kalech and R. Stern, "AI for Software Quality Assurance Blue Sky Ideas Talk," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 9, 2020, pp. 13634–13638. DOI: 10.1609/aaai.v34i09.7076.
- [20] D. Chanda, "AI Based Data Architecture Impact Analysis," *Proceedings of the International Conference on Software and Systems Engineering*, 2019, pp. 1–8. DOI: 10.29007/fkhl.

- [21] M. Kazemitabaar *et al.*, "Improving Steering and Verification in AI-Assisted Data Analysis with Interactive Task Decomposition," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, New York, NY, USA: ACM, Oct. 2024, pp. 1–19. doi: 10.1145/3654777.3676345.
- [22] S. Basri, N. Kama, R. Ibrahim, and S. A. Ismail, "A change impact analysis tool for software development phase," *International Journal of Software Engineering and Its Applications*, vol. 9, no. 9, pp. 193–212, Sep. 2015. DOI: 10.14257/IJSEIA.2015.9.9.21.
- [23] M. Borg, J. Vara, and K. Wnuk, "Practitioners' Perspectives on Change Impact Analysis for Safety-Critical Software - A Preliminary Analysis," in *Lecture Notes in Computer Science*, vol. 9826, 2016, pp. 404–416. DOI: 10.1007/978-3-319-45480-1_28.
- [24] N. L. Corvelo Benz and M. Gomez Rodriguez, 2024, "Human-Aligned Calibration for AI-Assisted Decision Making."
- [25] S. Massoni and N. Roux, "Optimal Group Decision: A Matter of Confidence Calibration," *Journal of Mathematical Psychology*, vol. 80, pp. 70–82, Aug. 2017. DOI: 10.1016/J.JMP.2017.04.001.
- [26] V. Vasudevan, A. Sathy, and A. R. Ghias, "Towards Better Confidence Estimation for Neural Models," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2019, pp. 7345–7349. DOI: 10.1109/ICASSP.2019.8683359.
- [27] G. Kurtz *et al.*, "Strategies for Integrating Generative AI into Higher Education: Navigating Challenges and Leveraging Opportunities," *Educ Sci (Basel)*, vol. 14, no. 5, May 2024, doi: 10.3390/educsci14050503.
- [28] S. Krishnan, D. Haas, M. Franklin, and E. Wu, "Towards reliable interactive data cleaning: a user survey and recommendations," in *Proceedings of the 2016 International Conference on Management of Data*, San Francisco, CA, USA, Jun. 2016, pp. 1–10. DOI: 10.1145/2939502.2939511.
- [29] A. Fatima, N. Nazir, and M. G. Khan, "Data Cleaning in Data Warehouse: A Survey of Data Pre-processing Techniques and Tools," *International Journal of Information Technology and Computer Science*, vol. 9, no. 3, pp. 47–56, Mar. 2017. DOI: 10.5815/IJITCS.2017.03.06.
- [30] A. Pant, R. Hoda, S. V. Spiegler, C. Tantithamthavorn, and B. Turhan, "Ethics in the Age of AI: An Analysis of AI Practitioners' Awareness and Challenges," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 3, Mar. 2024, doi: 10.1145/3635715.
- [31] A. P. Kharisma and M. S. Mardiyanto, "Towards text-based domain-specific modeling language for representational state transfer compliant services," in *Proc. ACM Int. Conf. on Informatics and Computing (ICIC)*, Nov. 2020, pp. 1–7. DOI: 10.1145/3427423.3427454.
- [32] T. Warnke and A. Uhrmacher, "Practical expressiveness of internal and external domain-specific modeling languages," in *Proc. Winter Simulation Conference (WSC)*, Dec. 2017, pp. 1–12. DOI: 10.1109/WSC.2017.8248207.
- [33] J. Kolluri, V. K. Kotte, M. PhridviRaj, and S. Razia, "Reducing Overfitting Problem in Machine Learning Using Novel L1/4 Regularization Method," in *Proc. Int. Conf. on Advances in Computing, Communication Control and Networking (ICACCCN)*, Jun. 2020, pp. 1–6. DOI: 10.1109/ICACCCN.2020.9142992.
- [34] M. S. Santos, J. Soares, P. Abreu, H. Araújo, and J. Santos, "Cross-Validation for Imbalanced Datasets: Avoiding Overoptimistic and Overfitting Approaches [Research Frontier]," *IEEE Computational Intelligence Magazine*, vol. 13, no. 4, pp. 59–76, Oct. 2018. DOI: 10.1109/MCI.2018.2866730.
- [35] N. Sherje, "Enhancing Software Development Efficiency through AI-Powered Code Generation," *Research Journal of Computer Systems and Engineering (RJCSE)*, vol. 5, pp. 2230–8563, 2024, doi: 10.52710/rjcse.90.
- [36] R. Ramler, M. Moser, L. Fischer, M. Nissl, and R. Heinzl, "Industrial Experience Report on AI-Assisted Coding in Professional Software Development," *Association for Computing Machinery (ACM)*, Apr. 2024, pp. 1–7. doi: 10.1145/3643795.3648377.
- [37] A. Habbal, M. K. Ali, and M. A. Abuzaraida, "Artificial Intelligence Trust, Risk and Security Management (AI TRiSM): Frameworks, applications, challenges, and future research directions," Apr. 15, 2024, *Elsevier Ltd.* doi: 10.1016/j.eswa.2023.122442.
- [38] C. Dolores Benitez and M. Serrano, "The Integration and Impact of Artificial Intelligence in Software Engineering," *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT) International Open-Access, Double-Blind, Peer-Reviewed, Refereed, Multidisciplinary Online Journal*, vol. 3, no. 2, 2023, [Online]. Available: <https://www.researchgate.net/publication/383455349>
- [39] L. Belzner, T. Gabor, and M. Wirsing, 2023, "Large Language Model Assisted Software Engineering," Springer. [Online]. Available: <https://www.researchgate.net/publication/374388078>
- [40] M. White, C. Vendome, M. Vásquez, and D. Poshyvanyk, "Toward Deep Learning Software Repositories," in *Proceedings of the 12th Working Conference on Mining Software Repositories (MSR)*, Florence, Italy, May 2015, pp. 1–11. DOI: 10.1109/MSR.2015.38.
- [41] R. Kapur and B. Sodhi, "Towards a Knowledge Warehouse and Expert System for the Automation of SDLC Tasks," in *Proc. International Conference on Software and System Processes (ICSSP)*, May 2019, pp. 1–10. DOI: 10.1109/ICSSP.2019.00011.
- [42] G. M. P. Wanderley, M.-H. Abel, J. Barthès, and E. Paraiso, "An Advanced Collaborative Environment for Software Development," in *Proc. IEEE*

International Conference on Systems, Man, and Cybernetics (SMC), Oct. 2016, pp. 1–6. DOI: 10.1109/SMC.2016.7844683.

- [43] K. Dam, T. Tran, J. Grundy, A. Ghose, and Y. Kamei, "Towards Effective AI-Powered Agile Project Management," in Proc. IEEE/ACM International Conference on Software Engineering (ICSE), May, 2019, pp. 1–9. DOI: 10.1109/ICSE-NIER.2019.00019.
- [44] N. Sundaresan, "From Code to Data: AI at Scale for Developer Productivity," in Proc. International Symposium on Code Generation and Optimization (CGO), Jul. 2019, pp. 1–10. DOI: 10.1145/3292500.3340410.
- [45] H. Jiang, "Challenges and Opportunities of Software Automation Discussed at the Yanqi-Lake Meeting," *National Science Review*, vol. 6, no. 2, pp. 1–12, Nov. 2018. DOI: 10.1093/NSR/NWY145.
- [46] A. E. Hassan, G. A. Oliva, D. Lin, B. Chen, M. (Jack, and) Jiang, 2024, "Rethinking Software Engineering in the Foundation Model Era: From Task-Driven AI Copilots to Goal-Driven AI Pair Programmers.
- [47] V. Dunjko and H. Briegel, "Machine Learning & Artificial Intelligence in the Quantum Domain," *arXiv preprint arXiv:1709.02779*, Sep. 2017. Available: <https://arxiv.org/abs/1709.02779>.
- [48] V. Dunjko and H. Briegel, "Machine Learning & Artificial Intelligence in the Quantum Domain: A Review of Recent Progress," *Reports on Progress in Physics*, vol. 81, no. 7, pp. 1–25, Jun. 2018. DOI: 10.1088/1361-6633/aab406.
- [49] N. Kama, S. Basri, S. A. Ismail, and R. Ibrahim, "Using Static and Dynamic Impact Analysis for Effort Estimation," 2019.
- [50] K. Sathyarajasekaran and R. Ganesan, "A Novel Process using Impact Analysis over Requirement Phase with Impact Transition Model," presented at the International Conference on Software Engineering, Dec. 2019.
- [51] R. Karra and A. Lasfar, "Analysis of QA System Behavior against Context and Question Changes," *The International Arab Journal of Information Technology (IAJIT)*, vol. 21, no. 2, pp. 191–200, Mar. 2024, doi: 10.34028/iajit/21/2/2.